



Procedural Level Generation in 2D Roguelite Games

Tomi Sepänmaa

Thesis
December 2024

Degree Programme in Business Information Systems
Games Production

TIIVISTELMÄ

Tampereen ammattikorkeakoulu
Tietojenkäsittelyn tutkinto-ohjelma
Games Production

SEPÄNMAA, TOMI:
Procedural Level Generation in 2D Roguelite Games

Opinnäytetyö 36 sivua
Joulukuu 2024

Tässä opinnäytetyössä tarkastellaan proseduraalista kentän generointia 2D-roguelite-pelien näkökulmasta. Työssä käsitellään pelien historiaa ja tarkastellaan, mistä ne ovat saaneet alkunsa sekä miten ne ovat ajan mittaan kehittyneet. Lisäksi selvitetään roguelite-peligenren synty sekä mitä se ylipäättään tarkoittaa.

Opinnäytetyön tarkoituksena on tehdä proseduraalinen generointi tutuksi sekä pohtia sen käyttötapoja. Työssä käydään läpi generoinnin etuja ja haittoja. Lisäksi tarkastellaan erilaisia algoritmeja, niiden toimintaperiaatteita ja käyttökohtia.

Opinnäytetyössä tutkittiin kolmea kaupallista roguelite-peliä ja niiden kentän generointia. Näiden pohjalta tehtiin spekulatioita millaisia, algoritmeja niissä on mahdollisesti käytetty ja mitä asioita koodissa mahdollisesti tapahtuu. Lisäksi haastateltiin Anssi Koskenrantaa, joka tarjosi arvokasta tietoa KILLBEAT-pelin kentän generoinnista.

Johtopäätöksenä todettiin, että proseduraalinen kentän generointi on tehokas työkalu, kun sitä käytetään oikeanlaisissa projekteissa ja tilanteissa. Generointi sopii erityisesti peleihin, joissa tarvitaan suurta määrää vaihtelevia kenttiä, mutta joiden sisältö pysyy kehittäjien hallinnassa. Generointi voidaan toteuttaa monella eri tavalla, joten pelin tarpeet on hyvä selvittää ennen sen toteutusta.

Jatkotutkimukselle aiheessa on paljon mahdollisuuksia, kuten olemassa olevien algoritmien jatkokehitys. Myöskään ei ole mahdotonta lähteä kehittämään täysin uudenlaista algoritmiä kentän generointiin. Lisäksi generatiivisen tekoälyn mahdollisuuksien tutkiminen osana kenttien proseduraalista generointia avaa isoja tutkimusmahdollisuuksia.

ABSTRACT

Tampereen ammattikorkeakoulu
Tampere University of Applied Sciences
Degree Programme in Business Information Systems
Games Production

SEPÄNMAA, TOMI:
Procedural Level Generation in 2D Roguelite Games

Bachelor's thesis 36 pages
December 2024

In this thesis, procedural level generation in 2D roguelite games is studied. The brief history of video games and where it all began is explored. Additionally, the birth of the roguelite genre is investigated, and it is examined how a game is defined as a roguelite.

The purpose of this thesis is to familiarize with procedural level generation and explore its use cases. Additionally, different kinds of algorithms, their processes and use cases are examined.

For this thesis, three commercial video games and their procedural level generation are analyzed. From these analyzes, speculations are made about which algorithms are used and how the code works. Additionally, valuable information about KILLBEAT's level generation was provided through an interview.

In conclusion, procedural level generation is a powerful tool if it is used in the right cases and projects. Procedural level generation is the right tool if enormous amounts of levels are needed, but the contents are controlled by the developers.

This topic offers broad possibilities for further research, such as further developing existing algorithms or even developing entirely new ones. Researching generative AI can also be a significant opportunity for procedural level generation.

Key words: Games, Game Development, Procedural Level Generation, Algorithm

TABLE OF CONTENTS

1	INTRODUCTION	6
2	HISTORY OF ROGUELITE	7
2.1	Brief History of Video Games	7
2.2	How “Rogue: Exploring the Dungeons of Doom” Was Born.....	8
2.3	Roguelike Genre	9
2.4	Roguelite Genre	10
3	WHAT IS PROCEDURAL GENERATION?	12
3.1	The Basics	12
3.2	Pros and Cons	13
3.2.1	Pros	13
3.2.2	Cons	14
3.3	Algorithms, Methods and More	15
3.3.1	Random Noise.....	15
3.3.2	Value Noise	15
3.3.3	Perlin and Simplex Noise	16
3.3.4	Fractal Noise	16
3.3.5	Seed	17
3.3.6	Drunkard’s Walk	17
3.3.7	Simple Room-Placement.....	18
3.3.8	Dog Leg Algorithm.....	18
3.3.9	Binary Space Partition	18
3.3.10	Cellular Automata	18
3.3.11	Voronoi Diagram	19
3.3.12	Delaunay Triangulation	20
3.3.13	Wave Function Collapse	21
4	PROCEDURAL GENERATION IN COMMERCIAL GAMES.....	22
4.1	Case: The Binding of Isaac: Rebirth.....	22
4.2	Case: Spelunky.....	25
4.3	Case: KILLBEAT	28
5	CONCLUSION	32
	REFERENCES	33

GLOSSARY

CPU	Computer Processing Unit
RNG	Random Number Generator
PRNG	Pseudorandom Number Generator
PCG	Procedural Content Generation
Run	One playthrough of game from beginning to the end or the death of the player character
Top-down	A game is presented from a bird perspective
Dungeon crawler	A game which core gameplay loop is to explore dungeons, kill monsters and collect loot
Permanent death	A game mechanic where all or some of the progression is lost and the game starts from the beginning when the player character dies
Permadeath	Same as permanent death
Unity	Video game engine by Unity Technologies

1 INTRODUCTION

The procedural generation is a powerful tool for creating large amounts of content in a game world. It can be used for almost anything, such as crafting unique non-player characters, stories, and missions. In the scope of this topic, procedural generation is studied in this thesis only from the perspective of level generation, more precisely from the perspective of the 2D roguelite game genre.

There are multiple algorithms to achieve procedural level generation. Some of these algorithms produce almost the same result, but differences might exist, such as in computing speed. Very often, multiple algorithms are used side by side to achieve the desired outcome. In this thesis, most of the algorithms are investigated superficially to understand how they work and what the common use cases are.

A couple of commercial games are studied to see how procedural level generation is handled and whether commonalities are shared in the process. The algorithms that are used are investigated.

The roguelite game genre is considered to predate the 1980s game *Rogue*, which combines procedural generation with storytelling. Every run in the game is made unique because of the procedural generation. Even the game developers do not know what happens during the walkthroughs.

2 HISTORY OF ROGUELITE

2.1 Brief History of Video Games

“OXO” was developed in 1952, and it is considered the world’s first video game. It was a tic-tac-toe game against the computer, which was running on the “EDSAC”, an Electronic Delay Storage Automatic Calculator. (Mikula n.d.)

The second breakthrough occurred in 1958 when physicist William Higinbotham invented a two-player game called “Tennis for Two”. It was running on an oscilloscope, and both players had their own controllers with a knob and a button. The angle of the ball could be changed with the knob, and with the button, the ball could be hit by the player. Tennis for Two did not have fancy graphics but it had gravity which affected the ball’s trajectory. The following year Higinbotham added a feature allowing the players to play in different gravities, such as the moon or Jupiter. (Tretkoff 22.8.2008)

In 1972, the first home console, the Magnavox Odyssey, was released. It used a game card system and even had a light gun, which was sold separately. A total of 28 games were available. (History 1.8.2017)

“The Strong National Museum of Play” refers to Pong as being the launcher of the video game industry. It resembled a tennis game from the Magnavox Odyssey but instead of being a game for the home systems, it was a coin-operated arcade machine. It started the arcade boom. (The Strong National Museum of Play 2015)

Nintendo started its video game business by releasing its first console called “Family Computer”, which was also known as “Famicom” in Japan in 1983. Later, that console was rebranded as the “Nintendo Entertainment System” or “NES” for the rest of the world. Games such as Super Mario Bros., The Legend of Zelda, and Metroid were released for the console, and are still highly praised and beloved video game franchises to this day. (Nintendo Wiki 28.7.2024)

In 1996, games with 3D graphics began to be favored over games with 2D graphics. That year, titles such as “Quake”, “Crash Bandicoot”, “Resident Evil”, “Tomb Raider”, and “Super Mario 64” were released. Due to hardware limitations, the games used different techniques to achieve good performance; for example, pre-rendered backgrounds were used in “Final Fantasy VII”. (Kiro talks 16.5.2023)

The video game industry is becoming increasingly larger. In 2023, the video game industry’s revenue was reported to be 406.2 billion U.S. dollars globally, and it is expected to grow by 46.43 percent between 2024 and 2029 (Statista 2024)

2.2 How “Rogue: Exploring the Dungeons of Doom” Was Born

Rogue is a brutal top-down dungeon crawler where procedural generation is used to create its levels and events. ASCII characters are used to visualize the player, items, and all the content in the game instead of fancy graphics. (TV Tropes n.d.)

In 1980, the most common home computers were the Commodore 64, the Atari 400 or 800, and Apple II. So-called “dumb terminals”, which had no graphical capabilities, were used by most students in universities. At the same time, a text-only role-playing game “Adventure” was a popular game on college computers. The game described what was happening in the game, and commands had to be typed by the player for the main character to follow. (Wichman 1997)

A library called “curses” was developed by Ken Arnold, a student at U.C. Berkeley, which allowed programs to put characters at a specific location on the computer screen. The curses library allowed “graphical” content to be created by developers by using letters, numbers, and symbols. (Wichman 1997)

The curses library began to spread around universities, and it was picked up by Michael Toy, along with Glenn Wichman, students at Santa Cruz, California. They liked playing “Adventure” and with the power of curses, a “graphical” adventure of their own was now possible to be created. Almost all of the code was written by Toy, and the name of the game, Rogue, was come up with by Wichman. Later,

Ken Arnold joined the development team and Glenn Wichman left for logistical reasons. (Wichman 1997)

The game was wanted to be develop so the creators could enjoy it too. The common problem with games during that time was that a game's content was handcrafted, and the solutions to puzzles and map layouts were known by the developers. The solution was given as: "The program itself should '*build the dungeon*', giving you a new adventure every time you played, and making it possible for even the creators to be surprised by the game". (Wichman 1997)

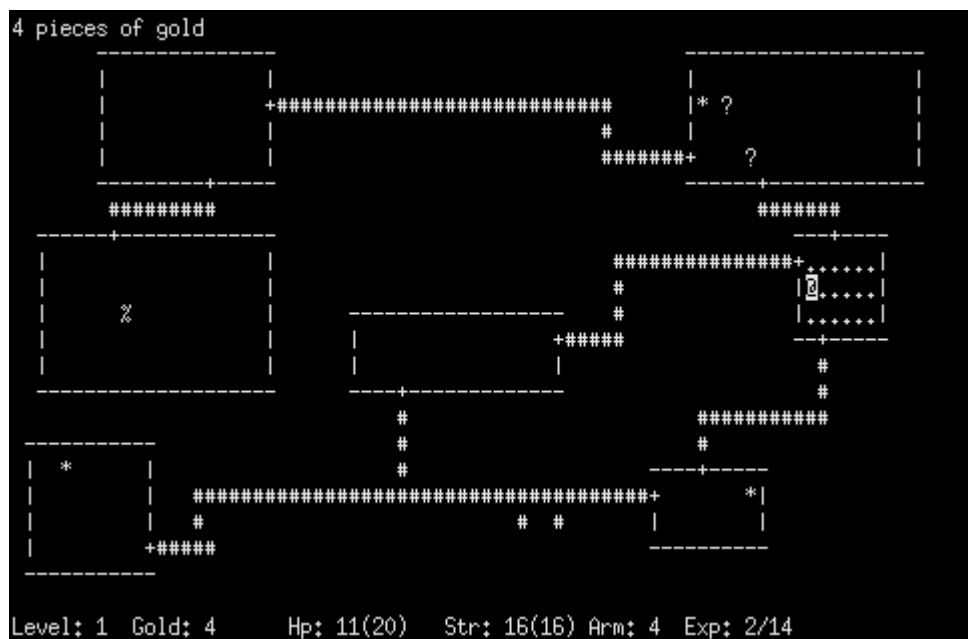


IMAGE 1. Screenshot from Rogue BSD version (Image by: Creative Commons Attribution-Share Alike 4.0, Thedarkb, 2021).

Later, Rogue was included in Berkeley's popular UNIX operating system version 4.2 BSD UNIX. Because of that, Rogue spread to universities around the world and became the most popular game on the college campuses. (Wichman 1997)

2.3 Roguelike Genre

The Rogue's popularity spawned a new genre and a rising wave of games that reminded the Rogue itself. Many of these clones or variants have been given their own variations. Some of the well-known games are Hack (1982), NetHack (1987), Moria (1983), and Angband (1990). (Intel 23.7.2019)

The use of the name “roguelike” began around 1993. A discussion labeled as “Request for Discussion” was started Andrew Solovay on the Usenet news.group (Zapata 13.11.2017), which had originally been created for message and file sharing between two universities (Andriekutė 3.1.2024). In the request, Solovay suggested a new hierarchy, `rec.games.dungeon` be created for the “rogue-type” games that shared the same characteristics. The games that were wanted to group there included *Rogue*, *Hack*, *Nethack*, *Moria*, and *Angband*. It was also pointed out that the name “dungeon” in the `rec.games.dungeon` might be misleading and it should be called `rec.games.roguelike` since the name was derived from the grandfather of these types of games. (Zapata 13.11.2017)

In 2008, a 15-point list called the “Berlin Interpretation” was made by attendants at the International Roguelike Development Conference to define what the “roguelike” genre is. The list was divided into high and low value factors. Even though a game does not check all the points on the list, it does not mean it is not considered roguelike and conversely, if it checks some of the points on the list, it does not mean it is roguelike. “The purpose of the definition is for the roguelike community to better understand what the community is studying. It is not to place constraints on developers or games” is stated in the Berlin Interpretation. The high values are: “random environmental generation”, “permadeath”, “turn-based”, “grid-based”, “non-modal”, “complexity”, “resource management”, “hack’n’slash” and “exploration and discovery”. The low value factors are: “single player character”, “monsters are similar to players”, “tactical challenge”, “ASCII display”, “dungeons”, and “numbers”. (Rogue Basin 24.8.2024)

2.4 Roguelite Genre

Roguelike and roguelite are often used as synonyms, but they are not. A lot of common features are shared between them, such as permadeath, procedural content generation, and character progression. The biggest difference between these two is that in roguelites, some of the progress is carried to the next run. This reward system allows the player to feel stronger and to be better prepared for the next run. (Switzer 26.4.2021)

The roguelite genre is considered much more open than the roguelike. Roguelite games usually have permadeath and procedural content generation, but other points from the Berlin Interpretation might be missing. (Switzer 26.4.2021) Roguelite games are often mixed with other genres as well. For example, “Crypt of Necrodancer” is categorized as a rhythm game, “Spelunky” as a platformer, and “Risk of Rain 2” as a third-person shooter. (Holmes 21.10.2024)



IMAGE 2. Screenshot from Risk of Rain 2 (Image by: Tomi Sepänmaa, 2024).

3 WHAT IS PROCEDURAL GENERATION?

3.1 The Basics

Procedurally generated means something is created by using some kind of algorithm instead of being loaded from memory. If a book opened or a song is started from a computer's hard drive, it is not procedurally generated. When discussing about generation, it does not mean it is truly random. "If the functions, expressions, algorithms, and inputs that are used to generate the content remain the same, then you'll always get the same results." says Dale Green. (Green 2016, 1. An Introduction to Procedural Generation.)

Computers are basically fancy calculators and hence cannot create true randomness. Every output made by a computer is the logical result of algorithms and equations. "Random number generation" or "RNG" for short, is still used in computers. To be more precise, it is "pseudorandom number generation" or "PRNG". Pseudo means not genuine, so fake random numbers are made by the computers. PRNG is used in many different things, such as simulations, games and cryptography. (Green 2016, Introducing randomness.)

It is stated that there is no such thing as true randomness in computer systems, or at least it is considered basically impossible to achieve (Watkins 2016, 1. Pseudo Random Numbers). With some extra equipment, it is possible, as a real-life wall of lava lamps is used by Cloudflare. Truly random movement patterns are produced by the lava lamps, which are fed into the system to create secure random number generation, or LavaRand as it is called. (Liebow-Feaser 6.11.2017)

"Seed" numbers are often used in games when random levels are generated. With the seed, the exact same level can be generated again and again. This helps the game to be debugged. The seed numbers can also be shared with others if the same level is wanted to be played or if the level is wanted to be rerun again. (Campbell 31.1.2020) For more information about seeds in ["Seed" chapter](#).

Basically, anything can be created by procedural generation in a game, and it is generally referred to as “Procedural Content Generation” or “PCG”. For example, it is used to create storylines in “The Elder Scrolls 5: Skyrim” or to generate items in the “Borderlands” series (Watkins 2016, Application of PCG).

Restrictions are often set at the start of procedural level generation. It needs to be thought about what is not wanted in a level, such as which enemy types are spawned at the beginning of the game and what different kinds of enemies are spawned in different biomes. (Short & Adams 2017, 14.)

3.2 Pros and Cons

3.2.1 Pros

Since the days of Rogue, content has been created by computers with limited memory and processing power using only calculations of procedural generation. The levels are not pre-made in the game’s file structure but are generated when needed, which saves memory and reduces the file size of the game. This unused free memory can then be used for other tasks. (Watkins 2016, Usage of PCG).

When a level is generated, a seed can be used for the generation rules. If the same seed is used, the generated level will always be the same. The same level can be generated again if the player decides to go back to the previous level even if the level was previously unloaded from the memory. (Watkins 2016, Random versus pseudo random numbers).

A virtually unlimited number of levels and variations can be had in a game when the game world is procedurally generated (Zenva 23.12.2023). For example, over 18 quintillion or exactly 18446744073709551616 planet possibilities are found in “No Man’s Sky” (No Man’s Sky Wiki 27.10.2024). Generating levels can save a lot of time as they do not have to be created by hand. Additionally, unique moments and surprises can also be given to players by the random element when the game cannot be predicted like another game that uses scripted moments. (Couture 3.5.2016) Higher replayability is had by the games with procedural

levels. The content is often considered to be the same yet different. (Short & Adams 2017, 9.)

Different kinds of rules and restrictions can be added to procedural level generation. In the book “Procedural Generation in Game Design”, it is stated that these are considered “restrictions of difficulty balance, connectivity, solvability of puzzles, and many other vital features”. (Short & Adams 2017, 9.) In the same book it is stated by Chris Chung that his game “Catlateral Damage” was restricted on a single floor: “Focusing on a single floor was entirely to keep level creation simple, specifically to not have to handle generating additional floors and the placement of stairs” (Short & Adams 2017, 59).

3.2.2 Cons

Every coin has two sides. If not planned properly a procedural level generator might become expensive and time-consuming. It is best for procedural generation to be include right at the start of the game project, not during the development. (Short & Adams 2017, 3.) Even though the game project is planned properly, procedural generation must be improved on the run, hence more time will be taken than traditional level design (Short & Adams 2017, 4).

It might be known how long it takes for 10 levels to be made for a game, but it is not known for sure how long it takes for a working level generator to be made. Wild guesses could be made about how long it will take to make one, but certainty cannot be achieved. (Short & Adams 2017, 7.)

The generated levels can be unbalanced, incoherent messes and/or boring to be play (Couture 3.5.2016). Sometimes serious bugs might be encountered in only 0,1% of the generated levels (Short & Adams 2017, 6). It is impossible to test every level before the game is shipped, and those levels might ruin the reputation of the game (Couture 3.5.2016).

It is considered a gamble to modify the level generator. If some great levels are made by the generator but the others are not so good, the possible fixes for the

generation rules might break the great ones at the same process. It is a dual-edged sword when the level generator is being enhanced. (Schier 23.10.2015)

If the room layouts are pre-handmade in the game but their amount and location are random generated the patterns can be seen by the players. It starts to be known how the rooms work, and the content starts to be predictable. (Donnelly 29.6.2015)

3.3 Algorithms, Methods and More

There are many different algorithms that can be used for level generation, and none of them is considered the best for every situation. Algorithms can also be combined to get the desired result that is needed by the game. Some of these algorithms are considered to be “Cellular Automata”, “Perlin Noise” and “Dunkard’s Walk” (Procedural Content Generation Wiki 6.1.2020). There are also so-called pathfinding algorithms such as “Breadth-First Search”, “Depth-First Search”, “Dijkstra” and “A*” which can also be used in level generation. For example, it is said by Brian Walker that a similar algorithm to Dijkstra was used to generate levels for his game “Brogue”. (Rogue Basin 11.9.2022)

3.3.1 Random Noise

A classic random noise which is familiar to many people as the “static” on old CRT televisions. Also known as “White Noise”. It is considered simple and chaotic. A random value is assigned to each point, and they are independent of each other. (Mujtaba 16.12.2023)

3.3.2 Value Noise

Value Noise can be used for level generation because it is considered more coherent and natural-looking than the White Noise (Mujtaba 16.12.2023). It can be generated from white noise by creating a grid that defines the edges of the cells and then interpolating the values at the grid points. The result is often quite rough, but it can be smoothed. (Suboptimal Engineer 26.4.2023) Value Noise is reminded of a zoomed-in White Noise (Mujtaba 16.12.2023).

3.3.3 Perlin and Simplex Noise

Perlin and Simplex Noise are almost the same. Both are developed by Ken Perlin, and Simplex is an improved version of Perlin (Perlin 2002, 681-682). Coherent pseudo-random noise is generated by Perlin Noise. A smooth gradual pattern is produced. If Perlin Noise is viewed as a one-dimensional, it is reminded of a wavelength. Two different main attributes are present: amplitude and frequency. The amplitude is the height of the wave along the y-axis, and the frequency determined by how often waves occur along the x-axis. (Lague 31.1.2016) The problem with this noise is that boring results are easily delivered when it is used in level generation. (White Box Dev 1.5.2021)

3.3.4 Fractal Noise

Fractal noise is Perlin noise, but it is multiplied and then combined. Mixing these noise maps or “octaves” with each other results in more natural looking levels. (White Box Dev 1.5.2021) In Sebastian Lague’s example video, three noise maps are combined. When the octaves are mixed together, a new variable called “lacunarity” is made, which increases the frequency of the maps to create fine detail in the result. Then a variable “persistence” is added to the mix, which is used to handle the amplitude of the maps. Without the persistence, the result would be looked rough but when it is added the result is made to look more natural. (Lague 31.1.2016)

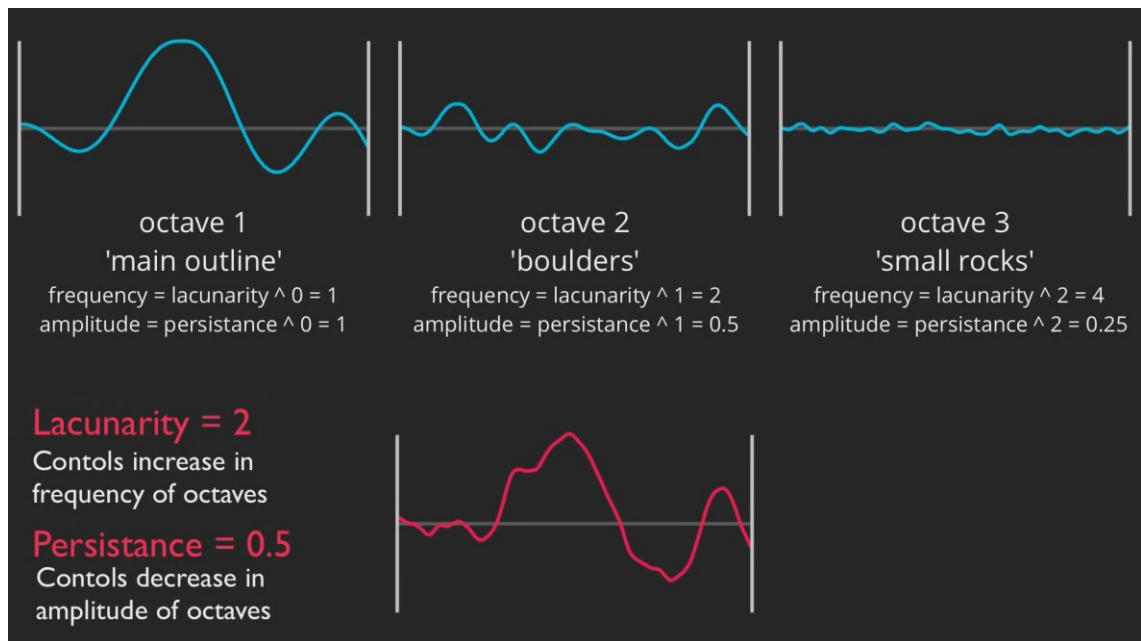


IMAGE 3. Octaves, lacunarity and persistence and their outcome presented in Lague's video (Image by: Sebastian Lague, 2016)

3.3.5 Seed

A seed is a value that is used for pseudo-random number generation. A predetermined sequence of numbers is produced by the PRNG based on a seed value. The starting position of this sequence is determined by the seed. From that point on, the "random" numbers are produced by a generator in a consistent order. As previously mentioned, using the same seed will always produce the same outcome because the sequence of the PRNG is predetermined. When a seed is generated automatically, it is often bound to the CPU clock. (Weimann 24.2.2024)

3.3.6 Drunkard's Walk

A really simple way to implement random level generation. A level is created with solid walls only, and a drunkard is spawned in there. A random number of steps is taken by the drunkard to any of four cardinal directions while the walls are smashed to the floor tiles. When the step count is full, a new drunkard could be spawned. This loop can be repeated as many times as is wanted and after that, the current floor is generated. (Wolverson 15.10.2020)

3.3.7 Simple Room-Placement

A very simple way to generate rooms into the level. First, the boundaries of the level are determined, and then it is filled with walls. Next, a rectangular room of random size is placed at a random location on the level. If that placement overlaps with another room, the latest room is destroyed, and another one is generated and placed. When all the rooms are generated, corridors between them are generated by using for example, a “Dog Leg” algorithm. (Wolverson 15.10.2020)

3.3.8 Dog Leg Algorithm

The “Gauss-Newton algorithm” and the “Steepest Descent method” are combined in this algorithm to check if the calculated steps are within a “trust region”. It is being used in non-linear least squares problems. First, a step is calculated with the steepest descent towards the target point. Next, it is checked if the step is inside the trust region by using Gauss-Newton algorithm. If the calculated step is outside the region, a new optimal step is determined by using Dog Leg path. This is blended from both Gauss-Newton and Steepest Descent and is kept inside trust region. (Wikipedia 24.9.2024)

3.3.9 Binary Space Partition

“Binary Space Partition”, or “BSP”, is delivered with really similar results to Simple Room-Placement, but the rooms are spaced out much better. In BSP, the map is divided horizontally or vertically into two, and the process is repeated on the new halves. This is done multiple times until remaining parts are about the size of a room. (Wolverson 15.10.2020) Random size rooms are generated inside the map parts and joined by corridors between each other. (Rogue Basin 15.8.2023) With the BSP, the rooms are never placed on top of one another (Wolverson 15.10.2020).

3.3.10 Cellular Automata

Cellular Automata is generated from a grid of cells that have two possible states, and these states can be, for example, dead or alive. (Morphocode n.d.) The state of each cell is depending on by its neighboring cells, which include adjacent cells in the up, down, left, right, and diagonal directions. When the cells' state is checked, a new grid is created and the cells' state check results are copied to the new grid. If the original grid is modified by the results, it will be filled with the same state because of the neighbors. (White Box Dev 5.11.2024)

In Cellular Automata a simple Noise Grid such as White Noise is often used as a base (White Box Dev 5.11.2024). In his "Procedural Map Generation Techniques" talk, it is addressed by Herbert Wolverson if a cell has one up to four neighbors, the cell becomes an empty, and if it has zero, five or more it becomes a wall. Not every Cellular Automata is the same and different rules for generation can be made. (Wolverson 15.10.2020) This process can be iterated over again and again, but the new grid must always be created where the new states of the cells are copied. (White Box Dev 1.5.2021)

3.3.11 Voronoi Diagram

The Voronoi Diagram is used to achieve organic-looking patterns, and it is found in many places in nature for example, on a giraffe's skin or the cells in a leaf. The basic principle is involved in dividing an area into smaller cells by calculating relations between random points. There are multiple algorithms to compute these cells. (Bellelli 22.2.2023)

Boundaries, known as perpendicular bisectors, are calculated between pairs of points to create cells. These bisectors are formed as edges of the cells, and this process is repeated with other points as well. Perpendicular bisectors are never crossed by each other. (Blue Shirt Khaki Pants 6.11.2024)

If the result is too random it can be improved with for example, "Lloyd's Relaxation" algorithm. Points are moved apart from each other by this algorithm and the result is more "rounder" and natural-looking shape. Lloyd's Relaxation can be iterated as many times as wanted. (Bellelli 22.2.2023)

3.3.12 Delaunay Triangulation

“Delaunay Triangulation gives us a set of triangles connecting all of our points together without overlap and avoiding really thin triangles as much as possible” is stated by Mike Bourgeois. When the triangles are calculated a circle called circumcircle is created, which passes through every three points. When there are no additional points inside the circumcircle the triangle is valid, and it is created. This loop is repeated until all the points are used to make triangles. (Bourgeois 9.6.2021)

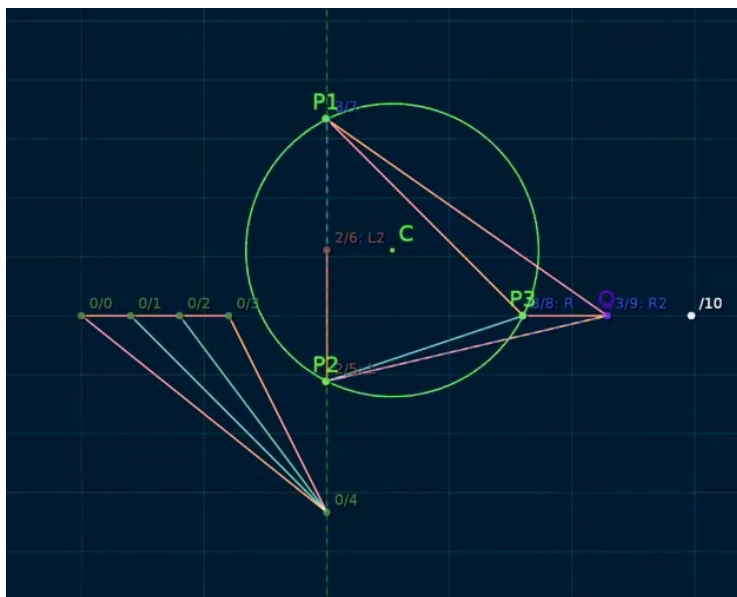


IMAGE 4. Delaunay Triangulation algorithm running on Mike Bourgeois YouTube video. A circumcircle is visible and it is running through points P1, P2 and P3 (Image by: Mike Bourgeois, 2021).

Delaunay Triangulation and Voronoi Diagrams are basically the same geometric computation, viewed from different perspectives. A Voronoi Diagram can be created from a Delaunay Triangulation, and vice versa. (Ledoux 18.12.2019) In Delaunay Triangulation, the vertices are the points from which the cells in the Voronoi Diagram are calculated. The center points of the circumcenters are located at the vertices of the cells in the Voronoi Diagram. Additionally, the edges of the triangles in the Delaunay Triangulation are made perpendicular to the edges of the Voronoi cells. (Bourgeois 9.6.2021)

3.3.13 Wave Function Collapse

Wave Function Collapse (WFC) is relatively new algorithm in procedural level generation, developed by Maxim Gumin and released as open source in 2016 (Bucklew 11.4.2022). The possible outcomes for an individual element are computed by the algorithm through relations between neighboring elements (Gumin 10.11.2024).

Input data such as pixel data, 2D tiles, 3D meshes, and so on are used from which constraints are derived. For example, when a texture with small dungeon layout with room and corridors is used, the dungeon of the same structure will be generated by the algorithm. This ensures that things like trees would not be placed on lakes or buildings would not be floated in the air. (Gumin 10.11.2024)

The element with the fewest valid possible outcomes, in other words with the smallest Shannon entropy, is resolved or “collapsed”. If two or more have the same entropy, one of them is randomly selected and collapsed. At the start each element is in a superposition of possible patterns which means that each point can collapsed into anything. When an element is collapsed, the superpositions of neighboring elements are updated. In practice, this means if a water tile is placed on a tilemap, its neighbours cannot be forest or mountain tiles. Loop is continued until all of the elements are collapsed. (Gumin 10.11.2024)

4 PROCEDURAL GENERATION IN COMMERCIAL GAMES

In this chapter, it is being studied how commercial games generate their levels and which or what kind of algorithms are being used. Unfortunately, the secrets have not revealed by all these developers. In these cases, the code of games has been reverse engineered, pseudo code has been created from there, or the way the level generation works has been studied by the people of online. Those sources of information are being used here too.

4.1 Case: The Binding of Isaac: Rebirth

The Binding of Isaac: Rebirth is one of the best known and highly acclaimed roguelites in the market. A huge influence has been made on the genre by it. (Holmes 21.10.2024) It is a remake of the original “The Binding of Isaac” game, which was created in Flash in 2011 (The Binding of Isaac Wiki 19.9.2023)

The game is divided into seven chapters, each containing different floors with unique themes, hazards and enemies. “The Basement” is reminded of a house, “The Womb” is revolved around blood, and “Flooded Caves”, like the name is implicated, is flooded with water. On one run, all the floor types are never visited by the player, and probably not the even chapters. (spider & Donia 25.10.2024)

The floors, also known as levels, are generated from a pool of rooms and their count on each level is determined by many factors: floor depth, difficulty level, whether the level has curses and whether the current level is “The Void”. The rooms come in different sizes and shapes. (The Binding of Isaac: Rebirth Wiki 2.11.2024) The room designs are a mix of random generation and handcraft and thousands of them are included in the game. Over 5000 new designs were added to the game by the third and the latest DLC “Repentance”. (The Binding of Isaac: Rebirth Wiki 10.11.2023)

It is stated by Boris in his blog that the level generation in Rebirth is based on the original The Binding of Isaac (Boris 12.9.2020). The first room is placed in the map and then randomly checked for all exits should there be a room next to it, as

stated by Florian Himsl who programmed the original game. If yes, a neighboring room is placed. This continues until the desired room count for the current floor is met. (Himsl 19.4.2020) This means that some kind of random walk algorithm is used when rooms are placed next to each other. There is also a check being made to ensure that the rooms do not overlap each other. It is also pointed out by Himsl that the rooms are empty, meaning that onlyt level layout is made in this phase (Himsl 19.4.2020).

The maximum floor size is 13x13 rooms, and bigger rooms cannot be partially placed outside of the limits in Rebirth (spider & Donia 25.10.2024). When the level layout is being generated, rooms are placed in branching lines, which means that normal 1x1 size rooms are not placed in clusters of four or more. Larger rooms, such as 1x2, 2x2 and L-shaped rooms tend to have this rule broken. Additionally, if the floor limits are being nearly filled and there are no available places for 1x1 rooms that fit within the standard rules, then they can be placed almost anywhere.

Six end rooms are created per floor at least, except that a minimum of five will be had in "The Basement". The special rooms, such as the boss, shop, and treasure room are generated to those end rooms. The special rooms have their own priority order in which they are generated. The boss room and secret rooms are always generated, and the others are subject to different changes and conditions to be generated into the current floor. The end rooms are added to a list or queue in descending order when the distance is calculated by rooms from the entrance. (The Binding of Isaac: Rebirth Wiki 10.11.2023) A check must be made to determine if the layout valid, and if it is not, it must be partly or fully discarded and then regenerated.

The special room types are next placed by the order of the end room queue, starting with the boss room. It is placed at the furthest end, and a super secret room is placed to the second furthest, and so on. Depending on the floor number and other variables the following rooms are placed to the dead ends in the following order: a shop, a treasure room, a planetarium, a dice or a sacrifice room, a library, a curse room, a mini-boss room, a challenge room, a vault or an arcade room, a bedroom, a normal secret room, an ultra secret room and last a grave

room. If all the end rooms are assigned a special, no more room special rooms can be placed. (The Binding of Isaac: Rebirth Wiki 10.11.2023)

When the room type assignment is ready the level layout is filled with the real rooms from the gigantic pool of premade designs. A check is included in the code to ensure that every room can be accessed by the player and that a soft-lock cannot occur. This can be achieved for example, by using a pathfinding algorithm like A star. If the player's path is blocked, the room's design is discarded, and another one is picked up. Of course, there are areas out of reach which can only be accessed by using tools or skills like bombs or by flying. These areas are not considered necessary for the progression but may contain optional content, such as a secret room or extra loot on the other side of the water.



IMAGE 5. A floor layout with a boss room (a skull), a treasure room (a crown), and a mini-boss (a smaller skull). The secret rooms are not found by the player and hence not visible. Other icons are loot on the ground in that room. (Image by: Boris, 2020)

When the room generation is completed, and the level is valid, the other content is generated into them. This means that enemies, bosses, items, doors, blood

donation machines et cetera are generated onto the level. Everything generated by the game is controlled by the seed number. If the same seed is used, everything is the same: the layout, the loot dropped by enemies, the bosses, and even the reroll of pedestal item.

The seed in *The Binding of Isaac: Rebirth* is 8 characters long, with “I”, “O”, “U” and “5” excluded and automatically replaced by “1”, “0”, “V” and “S”. Additionally, 79 special seeds are available, which give special effects for the next run and a random seed is automatically rolled for the normal level generation, with the exception for seeds that modify generation, such as endless Basement floors. (spider & Donia 25.10.2024)

4.2 Case: Spelunky

“Sometimes I’ll describe *Spelunky* as a platformer that’s inspired by ‘roguelikes’” is written in the book “*Spelunky*” by the game’s co-creator Derek Yu (Yu 2016, 4). It is a combination of platformer games, “Indiana Jones” movies and *Rogue*. Three key points are borrowed from The Berlin Interpretation of roguelikes: randomized level generation, permanent death and a shared ruleset of physical interactions between characters and items (Yu 2016, 6).

One of the key elements in the game is “caving” or “spelunking”, which is also implied by the game’s name “*Spelunky*” (Yu 2016, 29). Even though Yu never played, he was still heavily inspired by a Japanese game “*Kagirinaki Tatakai*” from 1983, which featured destructible environments (Yu 2016, 28). Destructible environments are given so much more immersion in a game and it also helped with the level generation: sections of the level are allowed to be generated out of the player’s reach (Yu 2016, 29).

The spelunking theme was suitable for the game for other reason too: it had previously been decided that the player must head downwards. Additionally, caves and dungeons are easy to create to look natural even when the levels are random generated and not handcrafted. The items that can be obtained by the player also fit with the spelunking theme: bombs, rope, and pickaxes. (Yu 2016, 30.)

“Spelunky has four different areas: the Mines, the Jungle, the Ice Caves, and the Temple” it is stated by Yu in his book. Each area contains four levels and totally 16 levels must be played through by the player to beat the game. (Yu 2016, 34.)

The levels in Spelunky are generated from rooms size of 10x8 tiles and they are placed in 4x4 room grid (Yu 2016, 34). Even though the rooms are being talked about the result is not looked at as a group of rooms but more like a maze. After all, it should be considered a cave or a dungeon not an apartment.

The level generator was wanted to have the following rules: 1. The entrance must be placed in the top row of the room grid. 2. The exit must be within reach without any special items. 3. The player should not be able to get stuck when any items are not being used. 4. The level should look seamless even though it is being created from 16 rooms on a grid. (Yu 2016, 34-35.)

First, one of the four spots from the top row of the level grid is picked up and marked as the entrance. Then, random movements are made left and right within the grid, and the lower level of the grid is reached either randomly or after an edge is hit. When moving horizontally, the spot is marked with a value of 1. If it is moved downwards, a value of 2 is given and a value of 3 is given if it moved from the upper row to that spot. This continues until the random walker reaches the bottom row and one of the grid spots is randomly marked as the exit. Every room that is not covered by the random walker is marked with a value of 0. (Yu 2016, 35-36.)

```

[ 0 ] [ X ] - [ 1 ] - [ 2 ]
               |
[ 0 ] [ 0 ] [ 2 ] - [ 3 ]
               |
[ 2 ] - [ 1 ] - [ 3 ] [ 0 ]
       |
[ 3 ] - [ 1 ] - [ X ] [ 0 ]
  
```

IMAGE 6. A level's room grid with added red lines to visualize how the random walker has made the level layout. (Image by: Derek Yu. 2016, modified)

With these values, it will be known what kind rooms should be assigned for each spot by the generator. The grid positions with 1 will have openings to the left and right. With the value of 2 an opening will also be placed downwards in a room, and with the value of 3 an opening will be placed upwards instead of downwards. The grid positions with the value of 0 are optional rooms and special items might be needed to get inside of those room. (Yu 2016, 35-36.)

When a room type value has been assigned to all grid positions, the rooms can be generated. A template for a room is randomly picked up by the generator, of which there are, depending on the room type, from six to twelve. “Templates are based on themes such as ‘basic room,’ ‘low ceiling,’ ‘ladders,’ ‘upper platforms,’ or ‘treasure below’” is stated by Yu. At first glance, room templates are seen as a bunch of random characters in a string such as “00000000110060000L040000000P110000000L110000000L1100000000110000000111112222111”. When line breaks are added after every 10th character, a part of a dungeon from a roguelike is reminded. (Yu 2016, 37.)

```
0000000011
0060000L04
0000000P11
0000000L11
0000000L11
0000000011
0000000011
1112222111
```

IMAGE 7. A room template string with line breaks (Image by: Derek Yu, 2016, modified)

A character is used represented each room tile. “0” is an empty space, “1” is a wall, “L” is a ladder, “P” is a ladder with a wooden platform, “4” is a pushable stone, and “2” has a 50% change of being a wall or empty space. “6” is an

exception. It does not represent a single tile but a group of 5x3 tiles. This group is randomized from its own pool of “chunks” and “6” is replaced in the template with it. (Yu 2016, 37-38.)

```

0000000011
0001110L04
0002220P11
0000000L11
0000000L11
0000000011
0000000011
1112222111

```

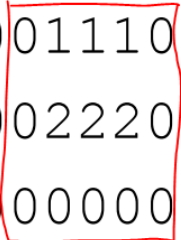


IMAGE 8. Position 6 replaced with a chunk in the room template. (Image by: Derek Yu, 2016, modified)

The idea behind the chunks is that more randomization is given to the game. The base of the room might look the same, but more variation is added when this kind of chunk system is used. (Yu 2016, 39.)

When the level is generated, it is populated with monsters, traps and loot. Every floor tile is checked by entity generator, and it decides whether an enemy spawns on it, next to it, or if it is skipped. Item spawn positions are located on tiles that are surrounded by three wall tiles. The spawn chance for every enemy, trap and loot is pretermined, and some are rarer than others. (Yu 2016, 39-40.)

4.3 Case: KILLBEAT

KILLBEAT is a rhythm-based top-down roguelite shooter from Tampere, Finland. It is being developed by a small indie company “Kalma Games”, and it is their first commercial product. KILLBEAT was initially started a couple years ago, and its

planned release is set for next year, 2025. The original idea comes from the CEO Atte Ronkanen, and the game takes inspiration from titles such as “Metal: Hellsinger” and “BPM: Bullets Per Minute”, “Enter the Gungeon” and “Risk of Rain”. (Koskenranta, 2024) For this chapter, Anssi Koskenranta from Kalma Games, who programmed KILLBEAT’s level generator, is interviewed.

Initially, there were probably no plans for what kind the level generation should be, but it was known that they wanted one. It was a time-saving tool for creating a lot of content for the game. Other games were studied how they approached level generation, but ultimately the generator that was made for the game was not a carbon copy of any other games’ level generator. During the development of KILLBEAT new features were added to the generator because the game’s idea evolved during the process and not everything had been planned from the start. (Koskenranta, 2024)

First, an entrance room of the corresponding floor is placed on the level by the generator. Next, the other rooms, except boss and exit, are randomly spawned from large pools of premade rooms. Each room type has its own pool with a spawn weight to be generated. Pools also have a limit on how many of each room type can be spawned per floor. Some rooms, such as a shop, are always generated. At this point, the rooms overlap with each other. (Koskenranta, 2024)

When all the rooms are generated, they will be pushed away from the entrance until they no longer overlap each other. After that, they will be drawn back towards the entrance to minimize gaps between them. The rooms are set to a grid and moved into a positive position so that a tile grid can be created from them. Some spacing is also added between the rooms, which helps later with corridors. Lastly, the exit room is being spawned somewhere on the edge of the floor, and the boss room is generated accordingly. (Koskenranta, 2024)

To generate corridors, all room entrances are first connected to each other by the level generator, while connections that overlap other rooms are avoided. Then the connections are reduced, ensuring that every room remains accessible. From there, a pathfinding algorithm is used to calculate their distance costs and paths with the lowest cost between two rooms are created. However, long corridors that

go around the rooms may still be generated. It is not intended, but they are accepted by the developers. (Koskenranta, 2024)

When asked what kind of challenges came up and how they were overcome, it is replied by Anssi that there was a lot of trial and error because it is hard to see what kinds of cases random generation brings. KILLBEAT uses seed and there are billions of possible levels, so it is impossible for every case to be tested. It had been joked previously that a computer should be put in the office, which generates levels and blacklists seeds that are invalid and sends that data to the players. The problem with this is that if even a minor tweak is made to the generator, the previously blacklisted seeds might no longer be invalid and the whole process would have to be started over again. (Koskenranta, 2024)

The common problems experienced with the level generator were that rooms were overlapping each other, no corridors were leading to a room, or two rooms linked with a corridor, but they were not connected to other rooms. Sometimes corridors were just spiraled. The level generation in general is tricky and weird, and unexpected edge cases such as corridors being formed into unintended patterns like a swastika, can easily be created. (Koskenranta, 2024)

In Unity, a special scene is used where the level generator can be stress tested. A seed and other configurations can be given to generate a specific level, or the number of random levels to be generated in a row, such as 100, can be specified. If an error is encountered, the generator stops, and error data can be examined from there. (Koskenranta, 2024)

No fail-safe mechanism is implemented in the level generator to “roll back” if an error is encountered. For example, rooms are not repositioned if errors occur during later steps of generation. If an error is encountered, the generation is restarted from the beginning. This could be looped a couple of times. (Koskenranta, 2024)

When asked about the estimated development time for the level generator, Anssi replied it would span the entire duration of the project. From the beginning, it was clear that the level generator must be tweaked through the whole development

and hopefully it would be in good shape when the game is released. (Koskenranta, 2024)

The level generator could be improved by reducing the empty space between the rooms, Anssi said when asked how he would do differently. Currently, the rooms' boundaries are square-shaped, and if a room's shape is for example, L-letter shaped there will be a lot of wasted space between the rooms. These boundaries could be calculated from multiple squares that cover the L-shape only and not the empty space next to it. (Koskenranta, 2024)

When creating a level generator, the more you have designed, the better he said when asked what to consider in advance. When you have clear goals what the generator should do, it is much easier to create than to add extra features during development. Even though it is very common for ideas to shift and changes must be made. (Koskenranta, 2024)

Generating levels is unique says Anssi. There are a lot of ready-made algorithms available on the internet that can be used for game development, but at the same time any of those would not probably fit in your game without tweaking. Every algorithm also has its own restrictions that must be taken into account. Another key point that Anssi highlights is performance: even if there is a good algorithm for your game, if the loading screen takes five minutes to generate your level it is way too much. Players would not wait that long. (Koskenranta, 2024)

5 CONCLUSION

Procedural level generation can be very powerful tool if designed well. In other hand it can be very cumbersome and time consuming. It can be virtually an endless void for money. For most of roguelite games procedural level generation can save time and money because it could make almost endless amount of content. Some games might work better if the content was handcrafted but often those games are from other genres. Good example is “Chasm” which represents “metroidvania” genre. The game’s problem is that the world is boring, repetetive and it feels really linear even though the world is procedurally generated.

Games using procedural level generation seem to follow the basic rule of generate, check and continue. The cases I studied followed that guideline pretty well by generating a part of level first then checked is everything valid, and if the result was valid the generator took the next step. There is no reason to generate a level on one try and check at the end is everything valid. It is only waste of time and computing resources.

This thesis was only a scratch on a surface on this topic. There are so many algorithms and use cases for them that I just could not include them here. Originally, I wanted to include more algorithms and write much deeper about them. I was also planning to write my own level generator, but I decided to drop it out for time reasons.

This topic has been interesting for me, and I was really surprised how easy to use some algorithms are. I think the level generation of Spelunky was the most surprising thing for me how much it can create for so little. For me this is just a launchpad for the procedural level generation and I want to read more of it and create that level generation of my own. The idea is to use it on some project wheather personal or commercial.

REFERENCES

- Andriekutė, A. 2024. What is Usenet? And how does it work in 2024?. NordVPN blog 3.1.2024. Referred 12.10.2024. <https://nordvpn.com/blog/what-is-usenet/>
- Bellelli, F. 22.2.2023. The Fascinating World of Voronoi Diagrams. Web page. Referred 6.11.2024. <https://builtin.com/data-science/voronoi-diagram>
- Blue Shirt Khaki Pants. 2020. Constructing Voronoi Diagrams. YouTube video. Released 18.5.2020. Referred 6.11.2024. <https://www.youtube.com/watch?v=j2c3kumwoAk>
- Bourgeois, M. 2021. Graphics! Voronoi, Delaunay, Natural Neighbor - Code, Sound & Surround E03. YouTube video. Released 9.6.2021. Referred 9.11.2024. <https://www.youtube.com/watch?v=jxOAU7YfypA>
- Boris. 2020. Dungeon Generation in Binding of Isaac. BorisTheBrave blog 12.9.2020. Referred 10.11.2024. <https://www.boristhebrave.com/2020/09/12/dungeon-generation-in-binding-of-isaac/>
- Bucklew, B. 2022. Tile-Based Map Generation using Wave Function Collapse in 'Caves of Qud'. YouTube video. Released 11.4.2022. Referred 10.11.2024. <https://www.youtube.com/watch?v=AdCgi9E90jw>
- Campbell, J. 31.1.2020. Procedural Generation and Randomness. Web page. Referred 29.10.2024. <https://unseendiplomacy2.com/2020/01/procedural-generation-and-randomness/>
- Couture, J. 3.5.2016. The pros and cons of procedural generation in Overland. Web page. Referred 29.10.2024. <https://www.gamedeveloper.com/design/the-pros-and-cons-of-procedural-generation-in-i-overland-i->
- Donnelly, N. 2015. Procedural Level Generation in The Dungeoning. Game Developer blog 29.6.2015. Referred 29.10.2024. <https://www.gamedeveloper.com/design/procedural-level-generation-in-the-dungeoning>
- Green, D. 2016. Procedural Content Generation for C++ Game Development. E-book. Birmingham: Packt Publishing Ltd. Referred 22.10.2024. Requires access. <https://learning.oreilly.com/library/view/procedural-content-generation/9781785886713/>
- Gumin, M. 29.4.2024. WaveFunctionCollapse. Web page. Referred 10.11.2024. <https://github.com/mxgmn/WaveFunctionCollapse>
- Himsl, F. 2020. Binding og Isaac: Room Generation Explained!. YouTube video. Released 19.4.2020. Referred 10.11.2024. <https://www.youtube.com/watch?v=1-HIA6-LBJc>

History. 1.8.2017. Video Game History. Web page. Referred 20.10.2024.
<https://www.history.com/topics/inventions/history-of-video-games>

Holmes, M. 21.10.2024. The Best Roguelike Games: the finest roguelites, deckbuilders, and RPGs. Web page. Referred 23.10.2024.
<https://rogueliker.com/best-roguelike-games/>

Intel. 23.7.2019. The History of the Roguelike in 23 Games. Web page. Referred 12.10.2024. <https://game.info.intel.com/gaming-access/the-history-of-the-roguelike-in-23-games>

kiro talks. 2023. The Birth of 3D Video Games. YouTube video. Released 16.5.2023. Referred 20.10.2024.
<https://www.youtube.com/watch?v=f7CU8FDwr2o>

Koskenranta, A. CTO of Kalma Games Oy. 2024. Interview 15.11.2024. Kalma Games.

Lague, S. 2016. Procedural Landmass Generation (E01: Introduction). YouTube video. Released 31.1.2016. Referred 5.11.2024.
<https://www.youtube.com/watch?v=wbpMiKiSKm8>

Ledoux, H. 2019. GEO1015 -- Triangulations & Voronoi diagram. YouTube video. Release 18.12.2019. Referred 9.11.2024.
<https://www.youtube.com/watch?v=ysLCuqcyJZA>

Liebow-Feeser, J. 2017. Randomness 101: LavaRand in Production. Cloudflare blog 6.11.2017. Referred 23.10.2024. <https://blog.cloudflare.com/randomness-101-lavarand-in-production/>

Mikula, L. n.d. OXO – The Birth of Computer Gaming. Web page. Referred 20.10.2024. <https://historyofcomputers.eu/timeline/oxo-the-birth-of-computer-gaming/>

Morphocode. n.d. Intro to Cellular automata. Web page. Referred 3.11.2024.
<https://morphocode.com/intro-to-cellular-automata/>

Mujtaba, M. 2023. Noise Functions. Gameidea blog 16.12.2023. Referred 4.11.2024. <https://gameidea.org/2023/12/16/noise-functions/>

Nintendo Wiki. 28.7.2024. Nintendo Entertainment System. Web page. Referred 20.10.2024. https://nintendo.fandom.com/wiki/Nintendo_Entertainment_System

No Man's Sky Wiki. 27.10.2024. Planets. Web page. Referred 13.11.2024.
<https://nomanssky.fandom.com/wiki/Planet>

Perlin, K. 2002. Improving noise. ACM transactions on graphics. 21 (3), 681-682. Released 1.7.2002. Referred 5.11.2024.
https://andor.tuni.fi/permalink/358FIN_TAMPO/176jdvt/cdi_proquest_miscellaneous_28970314

Procedural Content Generation Wiki. 6.1.2020. Map Generation. Web page. Referred 2.11.2024. <http://pcg.wikidot.com/pcg-algorithm:map-generation>

Rogue Basin. 11.9.2022. The Incredible Power of Dijkstra Maps. Web page. Referred 2.11.2024. https://www.roguebasin.com/index.php/The_Incredible_Power_of_Dijkstra_Map_s

Rogue Basin. 15.8.2023. Basic BSP Dungeon generation. Web page. Referred 3.11.2024. https://www.roguebasin.com/index.php?title=Basic_BSP_Dungeon_generation

Rogue Basin. 24.8.2024. Berlin Interpretation. Web page. Referred 12.10.2024. https://www.roguebasin.com/index.php/Berlin_Interpretation

Schier, G. 2015. Pros and Cons of Procedural Level Generation. Schier's blog 23.10.2015. Referred 29.10.2024. <https://schier.co/blog/pros-and-cons-of-procedural-level-generation>

Short, T. & Adams, T. 2017. Procedural Generation in Game Design. Boca Raton, Florida: CRC Press.

spider & Donia. 25.10.2024. The Binding™ - A Comprehensive Guide to Everything Isaac: Rebirth. Web page. Referred 12.11.2024. <https://steamcommunity.com/sharedfiles/filedetails/?id=1441000071&searchtext=A+Comprehensive+Guide+to+Everything+Isaac%3A+Rebirth>

Statista. 2024. Video game market revenue worldwide from 2019 to 2029. Web page. Referred 20.10.2024. <https://www.statista.com/forecasts/1344668/revenue-video-game-worldwide>

Suboptimal Engineer. 2023. What is Value Noise?. YouTube video. Released 26.4.2023. Referred 4.11.2024. <https://www.youtube.com/watch?v=KIIOFoUnKhU>

Switzer, E. 26.4.2021. Uh Oh, Everyone Is Mixing Up Roguelikes And Roguelites Again. Web page. Referred 13.10.2024. <https://www.thegamer.com/roguelike-roguelite-difference-hades-darkest-dungeon/>

Tretkoff, E. 22.8.2008. October 1958: Physicist Invents First Video Game. Web page. Referred 20.10.2024. <https://www.aps.org/apsnews/2008/09/first-video-game>

The Binding of Isaac Wiki. 19.9.2023. Welcome to the Binding of Isaac Wiki!. Web page. Referred 2.11.2024. https://bindingofisaac.fandom.com/wiki/The_Binding_of_Isaac_Wiki

The Binding of Isaac: Rebirth Wiki. 11.2.2024. Level Generation. Web page. Referred 2.11.2024. https://bindingofisaacrebirth.fandom.com/wiki/Level_Generation

The Binding of Isaac: Rebirth Wiki. 10.11.2023. The Binding of Isaac: Repentance. Web page. Referred 2.11.2024.

https://bindingofisaacrebirth.wiki.gg/wiki/The_Binding_of_Isaac:_Repentance

TheDarkB. 22.2.2021. File:Rogue Screenshot.png. Wikimedia Commons. Web page. Referred 4.12.2024.

https://commons.wikimedia.org/wiki/File:Rogue_Screenshot.png

The Strong National Museum of Play. 2015. Pong. Web page. Referred 20.10.2024. <https://www.museumofplay.org/games/pong/>

TV Tropes. n.d. Rogue (1980). Web page. Referred 20.10.2024.

<https://tvtropes.org/pmwiki/pmwiki.php/VideoGame/Rogue1980>

Watkins, R. 2016. Procedural Content Generation for Unity Game Development. E-book. Birmingham: Packt Publishing Ltd. Referred 22.10.2024. Requires access. <https://learning.oreilly.com/library/view/procedural-content-generation/9781785287473/>

Weimann, J. 2024. Made a Procedural World & Level Generator for my game (code in desc). YouTube video. Released 24.2.2024.

<https://www.youtube.com/watch?v=kliAhnHlKHc>

White Box Dev. 2021. Fractal Noise | Procedural Generation | Game Development Tutorial. YouTube video. Released 1.5.2021. Referred 5.11.2024. <https://www.youtube.com/watch?v=Z6m7tFztEvv>

Wichman, G. 1997. A Brief History of Rogue. Web page. Referred 30.9.2024.

http://www.digital-eel.com/deep/A_Brief_History_of_Rogue.htm

Wikipedia. 24.9.2024. Powell's dog leg method. Web page. Referred 3.11.2024.

https://en.wikipedia.org/wiki/Powell%27s_dog_leg_method

Wolverson, H. 2020. Herbert Wolverson - Procedural Map Generation Techniques. YouTube video. Released 15.10.2020. Referred 9.11.2024.

<https://www.youtube.com/watch?v=TILlOgWYVpI>

Yu, D. 2016. Spelunky. Los Angeles, CA: Boss Fight Books

Zapata, S. 2017. On the Historical Origin of the "Roguelike" Term. Santiago Zapata's blog 13.11.2017. Referred 12.10.2024. <https://blog.slashie.net/on-the-historical-origin-of-the-roguelike-term/>

Zenva. 23.12.2023. What is Procedural Generation - Complete Guide. Web page. Referred 29.10.2024. <https://gamedevacademy.org/what-is-procedural-generation/>